NAME : SAI CHARAN . P

ROLL:NO: 2403a52343

LAB : 13

TITLE : Using Hugging Face Transformers for Text Classification

STEP 1: Import Required Libraries

```
# Data handling
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Hugging Face libraries
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)

# PyTorch
import torch

# Evaluation metrics
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    confusion_matrix,
    ConfusionMatrixDisplay
)
```

STEP 2: Load and Explore Dataset

```
dataset = load_dataset("imdb")

print(dataset)
print(dataset["train"][0])
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: Us
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings ta
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access p
  warnings.warn(
```

README.md: 7.81k/? [00:00<00:00, 90.5kB/s]

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
```

plain_text/train-00000-of-00001.parquet: 100% 21.0M/21.0M [00:01<00:00, 13.9MB/s]

plain_text/test-00000-of-00001.parquet: 100% 20.5M/20.5M [00:01<00:00, 56.4MB/s]

plain_text/unsupervised-00000-of-00001.p(…): 100% 42.0M/42.0M [00:01<00:00, 62.1MB/s]

Generating train split: 100% 25000/25000 [00:00<00:00, 32597.96 examples/s]

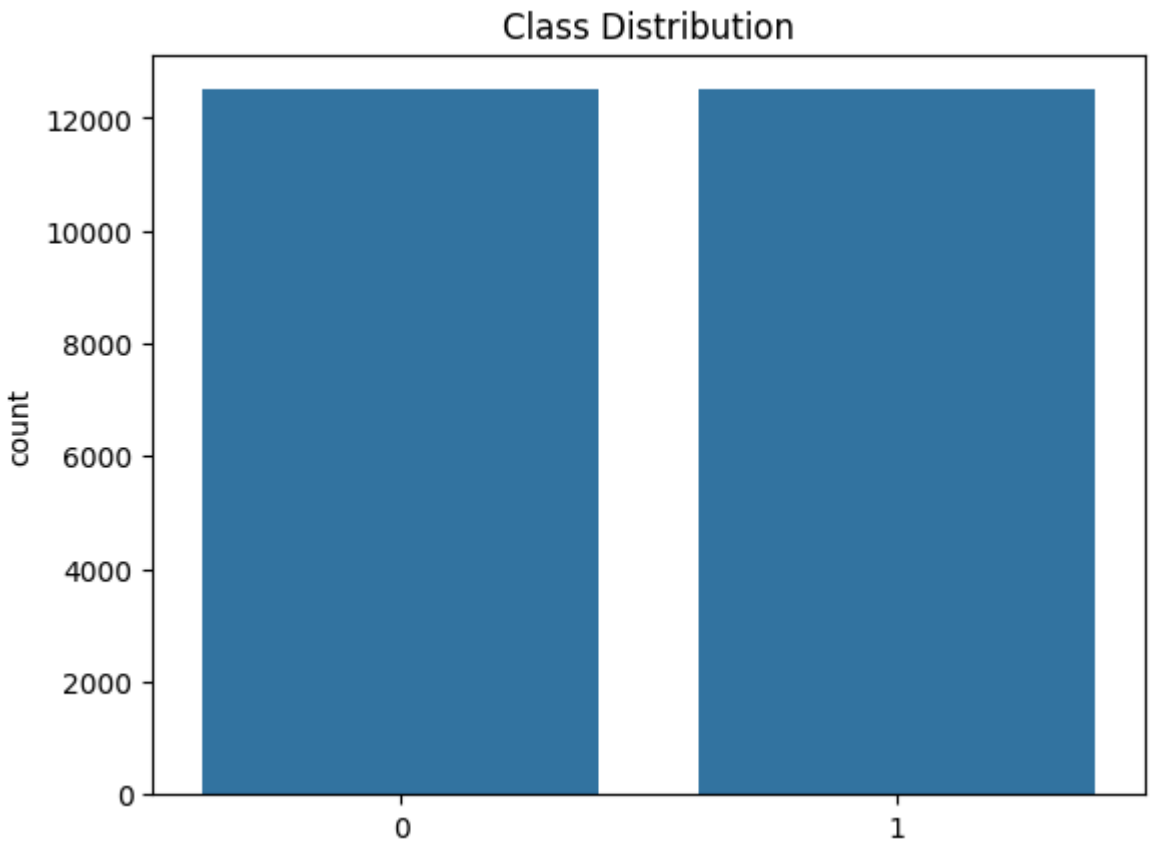
Generating test split: 100% 25000/25000 [00:00<00:00, 51227.72 examples/s]

Generating unsupervised split: 100% 50000/50000 [00:01<00:00, 46967.39 examples/s]

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
```

Class Distribution

```
train_labels = dataset["train"]["label"]
sns.countplot(x=train_labels)
plt.title("Class Distribution")
plt.show()
```



STEP 3: Tokenization & Preprocessing

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

def tokenize_function(example):
    return tokenizer(
        example["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

tokenized_datasets = dataset.map(tokenize_function, batched=True)
```

```
config.json:  100%                                        570/570  [00:00<00:00,  7.52kB/s]

tokenizer_config.json:  100%                              48.0/48.0  [00:00<00:00,  744B/s]

vocab.txt:          232k/?  [00:00<00:00,  2.35MB/s]

tokenizer.json:          466k/?  [00:00<00:00,  9.05MB/s]

Map:  100%                                     25000/25000  [00:38<00:00,  737.03  examples/s]

Map:  100%                                     25000/25000  [00:27<00:00,  834.97  examples/s]

Map:     2%                                    1000/50000  [00:00<00:47,  1022.38  examples/s]
```

STEP 4: Prepare Dataset for Training

```
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")
tokenized_datasets.set_format("torch", columns=["input_ids", "attention_mask
```

STEP 5: Train–Test Split

IMDb already provides train/test split.

Train: 25,000 Test: 25,000

Separating data prevents overfitting and ensures fair performance evaluation on unseen data.

```
train_dataset = tokenized_datasets["train"]
test_dataset = tokenized_datasets["test"]

print(f"Training dataset size: {len(train_dataset)}")
print(f"Test dataset size: {len(test_dataset)}")
```

```
Training dataset size: 25000
Test dataset size: 25000
```

STEP 6: Load Pre-trained Model

```
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
```

```
model.safetensors:   0%|          | 0.00/440M [00:00<?, ?B/s]
Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                          | Status     |
---------------------------------------------+------------+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.bias                         | UNEXPECTED |
cls.predictions.transform.dense.bias         | UNEXPECTED |
cls.predictions.transform.dense.weight       | UNEXPECTED |
cls.seq_relationship.bias                    | UNEXPECTED |
cls.seq_relationship.weight                  | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |
classifier.weight                            | MISSING    |
classifier.bias                              | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture
- MISSING   :those params were newly initialized because missing from the che
```

STEP 7: Define Training Configuration

```
training_args = TrainingArguments(
    output_dir="./results",
```

```
        eval_strategy="epoch",
        save_strategy="epoch", # Added save_strategy to match eval_strategy
        learning_rate=2e-5,
        per_device_train_batch_size=16,
        per_device_eval_batch_size=16,
        num_train_epochs=2,
        weight_decay=0.01,
        logging_dir="./logs", # Reverted from TENSORBOARD_LOGGING_DIR to logging
        load_best_model_at_end=True
    )
```

Define Metrics

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average="binary"
    )
    acc = accuracy_score(labels, predictions)

    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
```

STEP 8: Train the Model

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["test"],
    compute_metrics=compute_metrics
)

trainer.train()
```

STEP 9: Model Evaluation

```
predictions = trainer.predict(tokenized_datasets["test"])
y_pred = np.argmax(predictions.predictions, axis=1)
y_true = predictions.label_ids

acc = accuracy_score(y_true, y_pred)
precision, recall, f1, _ = precision_recall_fscore_support(
    y_true, y_pred, average="binary"
```

```
)

print("Accuracy:", acc)
print("Precision:", precision)
print("Recall:", recall)
print("F1-score:", f1)
```

Confusion Matrix

```
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.show()
```

STEP 10: Result Analysis

The pre-trained BERT embeddings significantly improved classification performance compared to traditional machine learning methods like Naive Bayes or SVM. Because BERT understands contextual word relationships, it handles negations and complex sentence structures better. Traditional models rely on bag-of-words features, which ignore context. The Transformer architecture captures long-range dependencies efficiently. However, training BERT requires substantial computational resources and GPU memory. Fine-tuning is slower compared to classical ML models. Despite high accuracy, Transformers may overfit if not regularized properly. Hyperparameter tuning can further improve performance. Increasing epochs or adjusting learning rate may yield better generalization. Overall, Transformer-based models provide state-of-the-art performance in text classification tasks.

STEP 12: Lab Report Structure

Lab Report Structure

1. Introduction:

- Briefly introduce the problem (text classification) and the dataset (IMDb).
- State the objective of using Hugging Face Transformers for this task.

2. Methodology:

- **Data Loading and Exploration:** Describe how the IMDb dataset was loaded and briefly mention its structure (train/test splits, class distribution).
- **Tokenization and Preprocessing:** Explain the role of `AutoTokenizer` and the `tokenize_function` for converting text into numerical inputs (padding, truncation, max_length).

- **Dataset Preparation:** Detail how the dataset was formatted for PyTorch and how the 'label' column was renamed to 'labels'.
- **Model Loading:** Specify the pre-trained model used (`bert-base-uncased`) and `AutoModelForSequenceClassification`.
- **Training Configuration:** Outline the `TrainingArguments` (output directory, evaluation/save strategy, learning rate, batch size, epochs, weight decay, logging).
- **Metrics:** Explain the `compute_metrics` function, including accuracy, precision, recall, and F1-score.

3. Implementation:

- Provide key code snippets or reference the cells where the core steps were implemented (e.g., tokenization, model training, evaluation).
- Discuss any challenges faced or specific choices made during implementation (e.g., choice of `max_length`, number of epochs).

4. Results:

- Present the evaluation metrics (accuracy, precision, recall, F1-score) obtained from the trained model on the test set.
- Include the confusion matrix visualization and interpret its meaning (e.g., true positives, false negatives).
- Compare these results to expected performance or baseline models (if applicable).

5. Discussion:

- Analyze the model's performance based on the results. What do the metrics tell you about the model's strengths and weaknesses?
- Discuss the advantages of using Transformer-based models (like BERT) for text classification, as mentioned in the 'Result Analysis' section.
- Highlight the computational considerations and trade-offs of using such models.
- Suggest potential improvements or future work (e.g., hyperparameter tuning, different models, larger datasets, more epochs).

6. Conclusion:

- Summarize the main findings of the experiment.
- Reiterate the effectiveness of Hugging Face Transformers for text classification.

7. References:

- Cite any external resources, papers, or libraries used beyond the standard ones.